

A 55-Symbol First-Order Set-Theoretic Definition of 2

GPT-5.6 Sol*

25 August 2026

Abstract

We give a 55-symbol formula in the austere first-order language of set theory that is satisfied by exactly the von Neumann integer $2 = \{\emptyset, \{\emptyset\}\}$. The symbol count is certified by a recursive parser, and a short uniqueness proof is given from Extensionality and Foundation. Under the convention that $\text{Rayo}(n)$ permits at most n symbols, the construction proves $\text{Rayo}(55) \geq 3$.

1 Definition of the language

We use variables as one-symbol tokens and the primitive formula grammar from Rayo’s presentation of satisfaction in first-order set theory [1]:

$$\phi ::= u \in v \mid u = v \mid (\neg\phi) \mid (\phi \wedge \psi) \mid \exists u(\phi).$$

Every variable, relation sign, connective, quantifier, and parenthesis counts as one symbol; whitespace is absent from canonical strings. Thus an atomic formula costs 3 symbols, negation and conjunction add 3, and existential quantification adds 4. The sole free variable below is x . Semantics are the standard, well-founded semantics of sets. We write $0 = \emptyset$, $1 = \{0\}$, and $2 = \{0, 1\}$.

The position of parentheses around negation is a presentation convention. Rayo’s grammar writes $(\neg\phi)$, while the initially discovered display used $\neg(\phi)$. Moving the opening parenthesis across $\neg$ preserves both meaning and length. The repository records the former as its canonical form.

2 The 55-symbol formula

The canonical string in `formula.txt` is

$$\exists a(\exists b(((a \in b \wedge (a \in x \wedge b \in x)) \wedge (\neg \exists c(\exists d((c \in d \wedge (d \in x \wedge (\neg a = c))))))))))$$

Its more readable, associatively faithful expansion is

$$\begin{aligned} \varphi(x) \equiv \exists a \exists b [& (a \in b \wedge (a \in x \wedge b \in x)) \\ & \wedge \neg \exists c \exists d (c \in d \wedge (d \in x \wedge a \neq c))] , \end{aligned}$$

where $a \neq c$ abbreviates the primitive subformula $\neg(a = c)$.

3 Symbol count

For a syntax tree T , define its length recursively by

$$\begin{aligned} L(u \in v) &= 3, & L(u = v) &= 3, \\ L((\neg\phi)) &= L(\phi) + 3, & L((\phi \wedge \psi)) &= L(\phi) + L(\psi) + 3, \\ L(\exists u(\phi)) &= L(\phi) + 4. \end{aligned}$$

The tree has six atomic leaves, five conjunctions, two negations, and four existential quantifiers. Hence

$$L(\varphi) = 6 \cdot 3 + 5 \cdot 3 + 2 \cdot 3 + 4 \cdot 4 = 18 + 15 + 6 + 16 = \boxed{55}.$$

The supplied `count.py` parses the canonical string, computes this recurrence, checks that the raw Unicode-code-point count agrees, and requires an exact serialization round trip.

*An OpenAI model; see <https://developers.openai.com/api/docs/models/gpt-5.6-sol>.

4 Uniqueness proof

Theorem 1. *In the standard universe of sets, $\varphi(x)$ holds if and only if $x = 2$.*

Proof. Suppose $\varphi(x)$, and choose witnesses a, b . The first conjunct gives

$$a \in b, \quad a \in x, \quad b \in x. \quad (1)$$

The last conjunct is equivalent to

$$\text{for every } d \in x \text{ and every } c \in d, \quad c = a. \quad (2)$$

Apply (2) to $d = a$, which is allowed by (1). If a had an element c , then $c = a$, so $a \in a$. Foundation excludes self-membership; consequently a has no elements, and Extensionality gives $a = \emptyset = 0$.

Apply (2) next to $d = b$. Every element of b is a , while (1) says $a \in b$. Therefore $b = \{a\} = 1$.

Finally let $d \in x$. By (2), all elements of d equal a . If d is empty then $d = a$. If d is nonempty, one of its elements must be a , and then Extensionality gives $d = \{a\} = b$. Thus $x \subseteq \{a, b\}$; (1) gives the reverse inclusion. Hence $x = \{a, b\} = \{0, 1\} = 2$.

Conversely, take $x = 2$, $a = 0$, and $b = 1$. Then (1) holds. If $d \in 2$, then d is either 0 or 1; every element of such a d , if any, is 0 = a . So the forbidden witnesses c, d do not exist, and $\varphi(2)$ holds. $\square$

Foundation is essential to this short argument: without well-foundedness, the condition on a would still permit a Quine atom satisfying $a = \{a\}$.

5 Consequences for Rayo(55) and the public baseline

Let $\text{Rayo}(n)$ denote the least natural number strictly larger than every finite natural uniquely named by a formula of at most n symbols in the language above. The theorem supplies a 55-symbol name for 2, so

$$\text{Rayo}(55) > 2, \quad \text{and therefore} \quad \boxed{\text{Rayo}(55) \geq 3}.$$

With a “fewer than n ” rather than an “at most n ” indexing convention, the identical statement is instead written $\text{Rayo}_{<}(56) \geq 3$.

As accessed on 25 August 2026, the Googology Wiki listed a 56-symbol formula as the current public string for 2 and recorded $\text{Rayo}(56) \geq 3$ [2]. The present construction improves that listed string by one symbol. No optimality claim is made here: ruling out all strings of length at most 54 is a separate problem.

6 Provenance and acknowledgements

The construction arose on 25 August 2026 in an interactive search with OpenAI’s GPT-5.6 Sol [3]. At the repository maintainer’s request, this public note is authored as GPT-5.6 Sol and does not name the maintainer. The model assisted with the search, proof checking, and preparation of the reproducibility files. This attribution records the requested provenance and does not imply endorsement by OpenAI. The Git history records the initial formula and subsequent verification revisions with explicit timestamps. Releases are archived through the GitHub–Zenodo integration, with a version-specific DOI assigned by Zenodo for each archived release.

References

- [1] A. Rayo, *Big Number Duel*, 2007. <https://web.mit.edu/arayo/www/bignumns.html>
- [2] Googology Wiki, *Rayo’s number*, section “Known values of Rayo’s function,” accessed 25 August 2026. https://googology.fandom.com/wiki/Rayo%27s_number
- [3] OpenAI, *GPT-5.6 Sol model documentation*, accessed 25 August 2026. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>